Effectiveness of a Generate–Verify–Repair Pipeline with Batfish for LLM-Generated Vendor CLI Configurations

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a preregistered paired experiment (cnsms2026-001-gvr-batfish-prereg-v1) that evaluated whether a generate–verify–repair (GVR) pipeline—shared initial LLM candidate (gpt-5-mini) + a deterministic Batfish-style validator + at most one automated LLM repair—reduces deterministic-verifier detected semantic/safety failures on intent→vendor-CLI tasks. Planned N=300 pairs; 266 complete pairs were analysed after preregistered exclusions. Baseline pass-rate = 260/266 (97.744%); guarded (final GVR) pass-rate = 265/266 (99.624%). Paired difference = 0.01879699248; exact McNemar two-sided p = 0.0625. Paired bootstrap (B=10,000, seed=1729) 95% percentile CI = [0.0037593984962406013, 0.03759398496240601]. Repair was invoked for 6 tasks; median repair iterations per task = 0. Per-episode latency was not recorded in per-task artifacts and thus could not be evaluated. Under shared_initial_candidate semantics the observed absolute improvement is small and did not meet the preregistered α=0.05 decision rule. We provide preregistered design details, deterministic validator semantics, exact paired counts, representative artifact-grounded repair examples, contamination findings (no exact public duplicates flagged), and a focused discussion of limitations and implications.**

I. Introduction

Generative large language models (LLMs) are actively explored for NetOps tasks such as intent→CLI translation, zero-touch configuration synthesis, and automated maintenance workflows [1], and surveys emphasize both opportunity and risk when LLMs are applied to operational network configuration [2]. Stochastic LLM outputs can produce syntactic or semantic violations—missing required commands, unsafe command orderings, or constraint breaches—that create operational risk if applied directly. To mitigate that risk, pipelines that interpose deterministic verifiers (e.g., Batfish or header-space analyses) and bounded repair loops—collectively generate–verify–repair (GVR)—have been proposed to provide deterministic acceptance criteria and automated correction [3,4]. Prior work presents components and prototypes, but preregistered, paired evaluations that archive verifier traces and quantify verifier-triggered repair benefit on reproducible NetOps task banks remain limited.

This study tested the preregistered question (cnsms2026-001-gvr-batfish-prereg-v1): Does a GVR pipeline that uses a single sampled initial LLM candidate per task (shared_initial_candidate semantics), a deterministic Batfish-style validator, and at most one automated LLM repair call reduce deterministic-verifier detected semantic/safety failures relative to the same initial candidate without repair? Our principal contribution is a preregistered, artifact-archived paired evaluation executed under the CNSM Agentic AI Researcher constraints that reports exact paired counts, inferential outcomes, execution accounting, representative artifact examples, and an evidence-grounded interpretation.

II. Related Work

Prior work relevant to this study spans (a) applied LLM prototypes and benchmarks for network configuration, (b) surveys documenting LLM failure modes and recommendations to ground generation, (c) proposals for generate–verify–repair and related architectures for deterministic validation, and (d) established deterministic network verification tools that serve as practical oracles in automated pipelines. We synthesize those threads and emphasize aspects most relevant to a preregistered paired GVR evaluation.

LLM-driven NetOps prototypes and benchmarks: Recent system-level and benchmark efforts investigate whether LLMs can aid network configuration generation and automation. NetConfEval and similar task collections examine intent→CLI translation and measure correctness under vendor-dialect constraints; these works provide public task exemplars and motivate reproducible task generators for controlled evaluation [1]. Such benchmark tasks typically encode required command sequences and vendor syntax, which directly map to the validator check types we archive in this study (required-sequence, allowed-field checks, topology reachability). Importantly, prior benchmarks often evaluate single-pass generation accuracy rather than integrated verifier-triggered repair, which motivates the paired GVR design adopted here.

Failure modes, grounding, and mitigation recommendations: Survey and synthesis papers summarize that LLMs commonly produce both syntactic and semantic errors when applied to domain-constrained tasks: missing commands, incorrect ordering, and hallucinated or unsafe modifications [2]. Those surveys recommend grounding strategies (retrieval, tool use), constrained generation, and verifier insertion to reduce the operational risk of accepting model outputs without deterministic checks. Our work adopts those prescriptions by interposing a deterministic validator and bounding the repair loop to one automated correction attempt, aligning experimentally with the practical mitigations advocated in the literature.

Generate–verify–repair and bounded repair proposals: The idea of interposing deterministic oracles and using closed-loop

repair is well established in proposals for regulated or safety-sensitive domains. Convergent GVR architectures aim to combine stochastic generation with deterministic acceptance criteria and limited repair to achieve practical correctness guarantees while preserving the generative model's productivity [3]. These proposals emphasize publishing verifier traces and repair prompts so that corrected outputs are auditable—an archival design we implemented in the execution artifacts (validator traces, repair_prompt_sha256, repair provider traces).

Deterministic network verifiers as oracles and their coverage limits: Tools like Batfish and header-space analyses provide deterministic, rule-based checks for reachability, ordering, and configuration invariants and have been incorporated into research prototypes that integrate model outputs with formal validation [4]. Batfish-style validators excel at expressing and mechanically checking reachability and required-sequence constraints on modeled topologies; however, their guarantees are strictly limited to the rule battery they encode. This distinction matters experimentally: a verifier that checks an explicit finite set of syntactic/semantic invariants yields a binary pass/fail endpoint that is reproducible and auditable, but it cannot detect semantic failures outside its coverage. Our deterministic_netops_validator_v5 follows this same engineering trade-off (syntactic parsing, required-sequence checks, allowed-fields enforcement, and topology reachability), and we archive per-episode validation_before/after traces so readers can inspect exactly which rules were exercised on each episode.

What LLMs need to synthesize correct router configurations and constrained decoding: Empirical studies analyzing LLM difficulty with router configuration point to sequencing, vendor-specific syntax, and dependency constraints as recurring challenges [5]. These analyses motivate two complementary mitigations: (1) stronger first-pass constraints or constrained decoding to reduce syntactic/format errors, and (2) verifier-triggered repair to correct semantic violations that slip through first pass. Our study deliberately fixes the initial candidate (shared_initial_candidate) and thereby isolates the marginal effect of the verifier-triggered repair step rather than conflating improvements due to constrained first-pass prompting or multiple draws.

Gaps in prior evaluations that motivated this study: The supplied literature shows active proposals and component evaluations but comparatively fewer preregistered, paired experiments that (i) use shared_initial_candidate semantics to measure the isolated effect of bounded repair, (ii) archive raw model traces together with deterministic validator traces and repair provider traces for each episode, and (iii) apply explicit exclusion, retry, and contamination rules in a confirmatory design. Those gaps motivated our preregistered paired design and the artifact-heavy archival choices (manifested in the execution and analysis artifacts we publish alongside this manuscript).

Methodologically relevant syntheses for experimental design: From the surveyed and prototyped prior work we derived three concrete design choices: (1) use a deterministic validator battery (Batfish-style) to obtain an auditable binary endpoint; (2) bound the repair loop (<=1 automated repair call per failed candidate) to maintain a clear paired estimand under shared_initial_candidate semantics; and (3) preregister explicit missingness, retry, and contamination treatments to avoid post hoc analytic choices. Each of these choices has precedent or justification in the cited literature: verifiers as deterministic oracles [4], GVR architectures for regulated settings [3], and benchmark/task construction recommendations that emphasize reproducibility and contamination controls [1,2].

In sum, our related-work synthesis locates this study at the intersection of applied intent→CLI benchmarks, survey-driven mitigation recommendations, GVR methodological proposals, and deterministic verification tooling. The specific empirical contribution is a preregistered, archived paired GVR evaluation under shared_initial_candidate semantics that fills a documented gap in artifact-grounded confirmatory assessments of verifier-triggered repair effectiveness.

III. Methodology

Overview and estimand. We preregistered the study as cnsms2026-001-gvr-batfish-prereg-v1 and analysed the paired estimand labeled paired_success_rate_difference_guarded_minus_baseline under shared_initial_candidate semantics. Under those semantics the same single sampled initial model candidate (per task) is used to compute both outcomes: baseline = validator pass/fail on the initial candidate; guarded = final validator pass/fail after the allowed validator-triggered repair (at most one LLM repair call). This design isolates the marginal effect of the permitted validator-triggered repair for a fixed initial candidate rather than conflating effects of different initial samples or different first-pass prompting strategies.

Task generation, inclusion/exclusion, and determinism. The preregistered task bank deterministically produced 300 intent→CLI pairs composed of public NetConfEval-style examples and synthetic tasks generated by a seeded deterministic generator. Tasks are stratified into two vendor-dialect clusters (150 Cisco-like, 150 Juniper-like). Generator IDs, seeds, and per-task payloads (initial_state, expected_final_state, required_sequence, allowed_touched_fields, and workflow_policies) are archived in the task manifest. Inclusion/exclusion rules and the retry policy followed preregistration: transient provider/API failures were retried up to two times; pairs were excluded from the primary confirmatory analysis only when both arms failed due to infrastructure/provider errors consistent with the preregistered missingness_plan. All task selection indices were fixed before model outcomes were observed and are archived to ensure deterministic replay of the task bank.

Model, orchestration and recorded runtime parameters. The declared model used in the run was gpt-5-mini (execution/model_configuration.json). Archived execution metadata records max_output_tokens = 1024, maximum_attempts_per_call = 1, provider =

openai_responses, and temperature = null (unset). Provider accounting in the deterministic reconciliation artifact records hosted_model_call_event_count = 306, completed_hosted_model_call_event_count = 272, and failed_hosted_model_call_event_count = 34; stage accounting shows shared_initial_generation = 300 and repair = 6. Because the provider configuration recorded temperature as unset and no centralized deterministic sampling seed is available for hosted calls, nondeterministic sampling of provider outputs cannot be excluded; we disclose this operational nondeterminism and treat it as a limitation in the Results and Disclosure.

Deterministic validator semantics and scoring rules. The binary oracle was deterministic_netops_validator_v5 and it applied a fixed battery of checks recorded in per-episode validator traces. For each candidate the validator (1) parsed and normalized the CLI commands (producing parsed_command_count and normalized_configuration), (2) checked the required_sequence field against parsed commands (required_command_count), (3) enforced constrained_touch policies (allowed_fields and allowed_touched_fields), and (4) executed reachability/constraint checks on the synthetic topology model to detect sequencing or dependency violations. The validator output includes observed_touched_field_count, violation_codes, and final valid boolean; the primary endpoint was the validator's binary pass/fail under the scoring rule full_intent_constraint_validation. Every episode archives both validation_before and validation_after traces so individual failure modes are inspectable and replayable.

Repair loop mechanics (bounded and archived). The preregistered repair stage is strictly bounded to at most one automated LLM repair call per task when the initial candidate fails the validator. The repair prompt construction is deterministic given the validator failure codes and the task payload: it includes the task constraints, the normalized initial candidate, and explicit violation codes, and it requests a corrected configuration that satisfies the validator battery. Repair provider traces and repair_prompt_sha256 values are archived when invoked; stage accounting reports six repair calls in the run. Repair outputs are submitted to the same deterministic validator and the guarded final outcome is the validator result after that single repair attempt. Limiting repairs to one invocation preserves a clear estimand and enforces a bounded repair budget as preregistered.

Analysis procedures and exact inferential specification. The prespecified primary inferential test is the exact McNemar two-sided test on paired binary outcomes at α=0.05. To quantify effect magnitude and uncertainty we also preregistered a paired bootstrap percentile confidence interval with B = 10,000 resamples and seed = 1729; those exact bootstrap parameters are recorded in analysis/analysis_log.jsonl. Missingness handling followed the preregistered complete_pair_primary rule: pairs where both arms failed due to provider infrastructure issues were excluded from the primary test (documented in analysis/missingness_summary.csv), and conservative sensitivity analyses treating excluded pairs as failures were preserved as archived artifacts.

Accounting, reconciliation and reproducibility artifacts. Execution accounting recorded planned_episode_count = 600 and completed_episode_count = 532, with failed_episode_count = 68; after applying preregistered exclusions the analysed complete_pair_count = 266. Deterministic reconciliation artifacts verify internal consistency of paired counts and provider calls (provider and stage counts, cache behavior, and cross-condition accounting) and assert that deterministic consistency checks passed. Key artifact categories archived for reproducibility include raw model responses, provider call traces, per-episode scoring JSONs (validator traces), the task manifest, transformation manifests, model_configuration.json, execution/raw_results.jsonl, and the analysis result artifacts (condition_summary.csv, paired_contingency_table.csv, missingness_summary.csv, contamination_summary.csv, and the paired_difference_figure.svg). File paths and manifests are provided with the execution bundle to enable exact reanalysis and targeted replay of repaired episodes.

Operational measurements attempted and absent. The preregistration included secondary estimands for median_repair_iterations_per_task and median end-to-end latency; repair iterations were measurable and recorded (repair calls = 6; median repair iterations per task = 0). Per-episode end-to-end latency (generation→validation→repair→final validation) was not recorded in the archived per-task artifacts and therefore the preregistered latency estimand could not be evaluated; this absence is noted as a preregistered deviation in the Limitations and drives a concrete reproducibility recommendation to include timing instrumentation in future runs.

Threats to validity embedded in the design. The methodology preserves explicit distinctions required by the preregistration: internal validity is governed by shared_initial_candidate semantics (the estimand measures repair benefit for a fixed sample rather than differences from alternative first-pass strategies), construct validity depends on validator rule coverage (the deterministic_netops_validator_v5 battery defines which semantic violations are detectable), and external validity is limited by the two vendor clusters and synthetic topology scale. The preregistration's missingness rules and sensitivity analyses were applied exactly and all exclusions, retries, and provider failures are archived in the execution manifest to support transparent effect-size interpretation.

IV. Results

Execution accounting and missingness: Planned episodes = 600; completed episodes = 532; failed episodes = 68 (execution_manifest.json). After applying preregistered exclusion rules (both-arm provider/infrastructure failures), the complete paired units analysed = 266 (analysis/missingness_summary.csv). Provider and model call accounting (deterministic_reconciliation.json) reports hosted_model_call_event_count = 306, completed_hosted_model_call_event_count = 272, failed_hosted_model_call_event_count = 34, cache_hit_count

= 0, cache_miss_count = 272, and stage_counts showing shared_initial_generation = 300 and repair = 6.

Primary confirmatory outcomes (complete_pairs = 266): baseline successes = 260; guarded final successes = 265. Paired contingency counts (analysis/paired_contingency_table.csv) are n11 = 260 (both pass), n10 = 5 (guarded pass only), n01 = 0 (baseline pass only), n00 = 1 (both fail). Observed proportions: baseline = 260/266 = 0.9774436090; guarded = 265/266 = 0.9962406015. Paired difference = 0.01879699248. Exact McNemar two-sided p = 0.0625 (preregistered α=0.05 decision rule $\rightarrow$ not significant). Paired bootstrap percentile CI (B=10,000, seed=1729) = [0.0037593984962406013, 0.03759398496240601] (analysis_log.jsonl archives bootstrap parameters and seed). The differing conclusions between exact McNemar (p>0.05) and the bootstrap CI (excludes zero) reflect small discordant counts and discrete p-value behavior: with few discordants (5 vs 0) exact McNemar's two-sided discrete p can be conservative while bootstrap CI summarizes resampled paired-difference variability.

Repair behavior and representative artifact examples: Repair stage calls recorded = 6; median repair iterations per task = 0. Representative episode: task-000008 shared initial candidate (execution/responses/task-000008-shared-initial.txt, sha256 = 89306095...) contained a truncated final token ("interface uplink2\n") and failed validation_before (parsed_command_count mismatch). The guarded final artifact (execution/responses/task-000008-guarded.txt, sha256 = d52b7616f...) contained the corrected sequence including the previously truncated line and the required edge1 commands; its validator trace (execution/scoring/task-000008-guarded.json) shows validation_after with valid = true and no violation codes. All repair invocations record repair_prompt_sha256 and repair_provider_trace_path in the archived scoring artifacts; these traces include the validator failure codes used by the repair prompt and the repaired configuration accepted by the validator.

Contamination and sensitivity analyses: The contamination detector (analysis/contamination_summary.csv) produced no exact public-duplicate flags; therefore no exact-duplicate tasks were excluded for contamination in the primary analysis. A preregistered sensitivity analysis that treats the 34 excluded both-arm episodes as failures (conservative complete N=300) yields baseline = 260/300 = 0.866667 and guarded = 265/300 = 0.883333 but retains discordant counts n10=5, n01=0 and exact McNemar p = 0.0625, indicating the primary inferential conclusion is robust to the preregistered conservative missingness handling.

Supplementary quantitative artifacts: analysis/condition_summary.csv and analysis/paired_contingency_table.csv are archived and hashed. deterministic_reconciliation.json documents that all deterministic consistency checks passed and provides provider call and stage counts used above. Per-episode validator traces (execution/scoring/*.json) provide normalized_configuration and explicit violation codes for every episode and permit replay or reanalysis of individual failure modes.

V. Discussion

The experiment produced a small absolute improvement in verifier pass-rate when the allowed repair stage could operate on a shared initial candidate: guarded - baseline = 0.01879699248 (n11=260, n10=5, n01=0, n00=1). Practically, this pattern—few discordant pairs with all discordance in the guarded$\rightarrow$pass direction—indicates that (a) the initial LLM candidate was correct for most tasks in this corpus, and (b) the bounded repair stage occasionally corrected defects that the initial candidate incurred. The artifact accounting (repair calls = 6; hosted_model_call_event_count = 306) confirms repairs were rare relative to total generation activity, explaining the modest absolute gain.

From a statistical perspective the run highlights an important small-sample inference nuance that directly affects operational interpretation. Exact McNemar (two-sided) returned p = 0.0625 under the preregistered α=0.05 decision rule, while a paired bootstrap percentile CI (B=10,000, seed=1729) produced a 95% interval that narrowly excludes zero. These outcomes are consistent: with very few discordant pairs (here 5 vs 0), the exact two-sided McNemar test is discrete and can be conservative; bootstrap resampling of paired differences summarizes empirical variability and can produce a CI excluding zero even when the exact discrete p slightly exceeds 0.05. For readers and practitioners this means that reporting both formal hypothesis outcomes and interval estimates (as archived here) gives a clearer picture: the effect magnitude is small but the CI suggests the effect is positive with narrow uncertainty in the observed corpus, whereas the exact hypothesis test under a strict α threshold does not claim formal rejection.

Operationally, artifact evidence supports three practical inferences grounded in this run. First, when first-pass generator accuracy is high ($\approx$97.7% baseline here), bounded automated repair offers limited marginal benefit because few failures remain to correct; this is exactly what we observed (6 repair calls on 300 tasks). Second, the marginal utility of GVR will be larger when (i) first-pass accuracy is lower, (ii) verifier coverage detects failure modes common in the target workload, or (iii) the repair mechanism is allowed more than one iteration—conditions not present in this bounded run. Third, the design of the verifier strongly conditions measured benefit: deterministic_netops_validator_v5 enforces a finite, archived battery of syntactic/sequence/constraint/reachability checks; any semantic failure mode outside that battery is neither detected nor credited to the GVR pipeline. In short, GVR's measured improvement depends both on how often the generator errs and on whether the verifier can detect the errors that matter operationally.

Threats to inference and external validity remain artifact-grounded. The preregistered shared_initial_candidate estimand isolates the effect of validator-triggered repair for a fixed initial sample; it does not measure improvements obtainable by engineering the first pass (e.g., constrained prompting, constrained decoding, or

multiple independent draws). Our execution manifest and reconciliation artifacts explicitly record stage_counts (shared_initial_generation = 300, repair = 6) and the lack of independent first-pass generations for separate arms, so extrapolating to comparisons that change initial sampling is unsupported by these artifacts. Missingness also reduced effective sample size: 34 both-arm provider failures were excluded per preregistered rules, lowering precision and power; deterministic_reconciliation.json and analysis/missingness_summary.csv archive these counts. Finally, per-episode latency was not recorded in per-task artifacts and therefore the preregistered latency secondary estimand could not be evaluated—this absence constrains operational claims about responsiveness or real-time applicability.

Design and research recommendations informed by these artifacts. (1) For stronger causal evidence about prompting or sampling strategies, future preregistrations should include additional independent arms that draw separate initial candidates (independent_generation semantics) or include constrained-first-pass prompting; these require separate initial model calls per arm and differ from the shared_initial_candidate estimand used here. (2) To assess operational trade-offs, pipeline runs must archive per-episode timing (generation→verification→repair→final validation) so latency and throughput can be measured; our provider accounting shows model call counts and failures but not per-episode latencies. (3) Because verifier coverage materially shapes measured benefit, future studies should expand the verifier rule set and publish clear catalogs of detectable failure codes; the per-episode validator traces archived in execution/scoring/*.json enable such audits and should be exploited to extend verifier coverage and to quantify verifier false negatives. (4) When baseline pass rates are high, consider targeting task corpora with intentionally lower baseline accuracy (e.g., out-of-distribution intents, noisier prompts, or reduced grounding) to increase discordant counts and thereby statistical power to detect repair benefits.

Concluding synthesis: the archived artifacts show that a bounded GVR pipeline can correct occasional deterministic-validator failures for a fixed initial candidate, but when the generator's first-pass accuracy is already high and the verifier's coverage is finite, the absolute operational improvement is small. The run's reconciled accounting, validator traces, repair provider traces, and contamination checks (no exact public duplicates flagged) provide a reproducible empirical substrate for these conclusions and for follow-on experiments that expand sampling semantics, verifier coverage, or repair budgets.

VI. Limitations

- Shared_initial_candidate semantics: both arms used the same single sampled initial candidate per preregistration; the estimand measures validator-triggered repair benefit for that fixed initial sample and does not evaluate independent first-pass generations or constrained first-pass prompting strategies.
- Missingness and sample reduction: planned N=300 pairs; after infrastructure/provider exclusions 266 pairs were complete and 34 both-arm episodes were excluded per preregistered rules, reducing effective sample size and precision.
- Small discordant counts: primary inference used exact McNemar with few discordants (n10=5, n01=0); conclusions are sensitive to inferential choice and limited power.
- Validator coverage: deterministic_netops_validator_v5 enforces a finite set of syntactic/semantic/reachability rules; semantic violations outside its coverage are possible and unmeasured.
- Runtime nondeterminism and sampling: archived execution/model_configuration.json records temperature = null and no deterministic seed; nondeterministic sampling cannot be excluded for recorded model calls.
- H2 latency not evaluated: per-episode end-to-end latency was not present in archived per-task artifacts and therefore the preregistered latency bound (median <= 60 s) could not be assessed.
- External validity: task corpus derives from public NetConfEval-style examples and a seeded synthetic generator limited to two vendor-dialect clusters; results do not imply universal benefit across other vendors, larger topologies, or production deployments.

VII. Conclusion

We executed a preregistered paired experiment that evaluated a GVR pipeline (gpt-5-mini + deterministic_netops_validator_v5 + <=1 automated repair) on intent→CLI tasks under shared_initial_candidate semantics. Across 266 analysed pairs the guarded pipeline improved validator pass rate by 0.01879699248 but did not meet the preregistered exact-McNemar α=0.05 decision rule (p=0.0625); a paired bootstrap CI (B=10,000, seed=1729) narrowly excludes zero. Repair calls were rare (6/300) and median repair iterations per task = 0. Per-episode latency was not archived and could not be assessed. All execution and analysis artifacts (raw responses, validator traces, provider call accounting, and reconciliation manifests) are archived in the evidence bundle to support reanalysis and follow-on work.

References [1] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?," in Proc. ACM, 2024, doi: 10.1145/3656296. [2] S. Y. Long, J. Tan, B. Mao, F. Tang, Y. Li, M. Zhao, and N. Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models," IEEE Commun. Surveys & Tutorials, 2025, doi: 10.1109/comst.2025.3526606. [3] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments," SSRN, 2026, doi: 10.2139/ssrn.6754899. [4]

M. Brown, A. Fogel, D. Halperin, V. Heorhiadi, R. Mahajan, and T. Millstein, "Lessons from the evolution of the Batfish configuration analysis tool," Proc., 2023, doi: 10.1145/3603269.3604866. [5] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to Synthesize Correct Router Configurations?," Proc., 2023, doi: 10.1145/3626111.3628194.

Disclosure Statement

Disclosure (concise): Declared model: gpt-5-mini (execution/model_configuration.json archived; declared_model_version = "gpt-5-mini"). Orchestration adapter: hosted_netops_gvr_v1. Deterministic validator: deterministic_netops_validator_v5. Preregistration: cnsms2026-001-gvr-batfish-prereg-v1 (manifest SHA-256: 46a223492f760950b3d06f475129ce21e52e296c4c2a1e09 df3ccd50bec9f891). Initial master prompt immutable reference: provenance/master_prompt.txt (SHA-256: 1872df1e 1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Human role and autonomy boundary: before the autonomy lock a human Research Director selected the topic family and approved pre-lock infrastructure development; after the autonomy lock the registered pipeline autonomously performed literature selection, preregistration, experimental execution, analysis, manuscript drafting, AI peer review simulation, and revision. Nondeterminism disclosure: archived execution/model_configuration.json records temperature = null (unset) and no deterministic sampling seed is archived; therefore nondeterministic sampling of model outputs cannot be excluded for the recorded provider calls. Execution and analysis artifacts, logs, and hash manifests are archived in the evidence bundle referenced in the preregistration.

References

[1] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296

[2] S.Y. Long, Jingjing Tan, Bomin Mao, Fengxiao Tang, Yangfan Li, Ming Zhao, Nei Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", 2025, 10.1109/comst.2025.3526606

[3] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899

[4] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, Todd Millstein, "Lessons from the evolution of the Batfish configuration analysis tool", 2023, 10.1145/3603269.3604866

[5] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194